Distributed prefix caching for LLM inference has transformed since RadixAttention

The single-node radix tree that SGLang introduced in December 2023 has spawned a full ecosystem of distributed KV cache systems, and the state of the art in early 2026 looks dramatically different. SGLang itself evolved RadixAttention into HiCache — a three-tier hierarchical caching architecture spanning GPU HBM, CPU DRAM, and distributed storage — while competitors like Mooncake (FAST 2025 Best Paper) (GitHub) built production-grade KVCache-centric disaggregated architectures processing **100+ billion tokens daily**. (ACM Digital Library) (USENIX) The field has converged on a layered stack: a prefix-aware inference engine (SGLang or vLLM), a KV cache middleware layer (LMCache), multi-tier storage with RDMA-based transfer (Mooncake, NIXL), and cache-aware request routing. Position-independent caching — reusing KV cache for text chunks regardless of their position in the prompt — represents the most significant frontier (OpenReview) beyond what RadixAttention originally proposed.

How RadixAttention works and what made it novel

SGLang's RadixAttention, introduced in arXiv 2312.07104, maintains a **radix tree** (compressed prefix tree) as an LRU cache of KV cache entries across all requests served by a single engine. (LearnOpenCV) (sugiv) Each tree node stores a token sequence key, pointers to KV cache pages in GPU memory, a reference count preventing eviction of in-use entries, and access timestamps. Unlike vLLM's fixed block-size PagedAttention, SGLang uses variable-length token sequences as edge labels, enabling automatic and fine-grained prefix sharing without manual configuration. (sugiv) (Medium)

The key innovation was **tree-structured sharing**. Where prior systems could only reuse KV cache for linear prefixes (system prompt → user query), RadixAttention automatically discovers and exploits multi-level sharing patterns: few-shot examples, Tree-of-Thought branches, self-consistency sampling, multi-turn chat histories, and agent tool-calling loops — all represented naturally in the radix tree. Cache-aware scheduling sorts queued requests by matched prefix depth, provably approaching **96% of optimal** cache hit rates online. In production on LMSYS Chatbot Arena, this achieved 52–74% cache hit rates and 1.7× first-token latency reduction.

SGLang's own evolution: from single-node tree to hierarchical distributed caching

SGLang has undergone five major architectural advances since the original paper, culminating in a system deployed on **400,000+ GPUs** generating trillions of tokens daily (GitHub) as of February 2026 (v0.5.9).

Eviction and cache specialization. The original LRU-only eviction now supports configurable **LRU and LFU** policies. The cache subsystem has also diversified into specialized implementations: (RadixCache) for general workloads, (ChunkCache) for long-sequence chunked prefill, sliding-window attention variants

(SWARadixCache), sugiv and MambaRadixCache for hybrid linear-attention models that maintain dual LRU lists for KV states and Mamba states separately.

HiCache (September 2025) represents the most significant architectural leap. It extends RadixAttention with a **HiRadixTree** — a page table referencing KV caches across three hierarchy levels inspired by CPU cache design. **L1** is GPU HBM for active computation. **L2** is host CPU DRAM (configurable, default 2× L1 size). **L3** is distributed storage shared across all inference instances in a cluster. Each node in the HiRadixTree records which tier stores its KV data. For L3, metadata is queried in real-time from the backend to reduce synchronization overhead. The system supports three write policies — write_through for maximum durability, write_through_selective that uses hit-count tracking to only back up hot data, and write_back for deferred writes. Custom GPU-assisted I/O kernels deliver **3× higher throughput** for CPU-GPU transfers. Performance results show up to **6× throughput improvement** and **80% TTFT reduction**; on a coding agent scenario with Qwen3-Coder-480B (25K+ token sessions), cache hit rates jumped from 40% to 80%.

HiCache's L3 tier supports pluggable storage backends: **Mooncake** (RDMA-accelerated distributed memory pool), **DeepSeek 3FS KVStore**, **NVIDIA NIXL** GitHub (unified API for various storage plugins), and **AIBrix KVCache**. GitHub This effectively transforms RadixAttention from a single-node mechanism into a cluster-wide distributed caching system.

Cache-aware routing arrived in v0.4 (December 2024) as the sgl-router, a Rust-based component that maintains an approximate radix tree mirroring each data-parallel worker's actual cache state, lazily updated with near-zero overhead. LMSYS Org When a request arrives, the router predicts prefix cache hit rates across workers and routes to the best match. This delivered **1.9× throughput** and **3.8× higher cache hit rates** over round-robin. By v0.3.1 of the Model Gateway (early 2026), the routing system achieved **10–12× performance improvement** and **99% memory reduction** in cache-aware routing operations — processing 474,000 concurrent routing operations per second across 64 threads.

Prefill-decode disaggregation separates compute-intensive prefill from memory-bound decode onto dedicated servers, Sglang with KV cache transferred via Mooncake or NIXL over RDMA. NVIDIA An experimental **Uni-PD** mode eliminates explicit KV transfers entirely — prefill kernels write KV cache directly into the decode node's HBM using NVSHMEM, demonstrated on GB200 with DeepSeek-R1 yielding **8.12% TTFT reduction** by eliminating 100% of explicit transfers. GitHub

---

Production systems compared: five approaches to prefix caching

vLLM: hash-based blocks, extended via middleware

vLLM's Automatic Prefix Caching (APC) uses **hash-based block matching** rather than a radix tree. vLLM Each KV cache block (typically 16 tokens) is identified by hash(prefix_tokens, tokens_in_block) using SHA-256 (default since v0.11). When a request arrives, the scheduler hashes its token blocks left-to-right and looks up a hash table. This is fundamentally a **linear prefix** caching mechanism — it stops at the first cache miss and does not natively support tree-structured sharing. Eviction is LRU-based via a doubly-linked free queue. vLLM

vLLM's APC operates **within a single engine instance only**. (vLLM) Cross-node distribution relies on external systems: (arXiv) **llm-d** (IBM/Google/Red Hat) maintains a global KV cache index via [KVEvents] streamed from each vLLM pod, (GitHub) achieving **57× faster TTFT** and **2× throughput** versus round-robin, (llm-d) with 87.4% cache hit rates. (Red Hat) **LMCache** provides a middleware layer enabling hierarchical caching across GPU → CPU → disk → remote storage (Lmcache) (GitHub) (Redis, Ceph S3, Mooncake Store) using hash-based block identification (Ceph) (Lmcache) with larger 256-token blocks for better amortization, delivering up to **15× throughput improvement**. (Lmcache) vLLM's pluggable [KVConnector] API supports Mooncake Transfer Engine and NIXL for disaggregated serving. (Kvcache-ai)

TensorRT-LLM: radix tree with priority-based eviction and early reuse

NVIDIA's TensorRT-LLM also uses a **radix search tree** for prefix caching, storing blocks as they fill and searching for matching prefixes when new requests arrive. (GitHub) Its most distinctive feature is **early reuse** — it can share system prompt KV cache *as it is being generated in real time*, enabling sharing during concurrent bursts with up to **5× TTFT improvement**, (NVIDIA Developer) something other systems cannot do since they require computation to complete before reuse.

Eviction uses a **priority-based** policy where users can specify priority and duration for discrete token ranges (e.g., system prompt at maximum priority). The system traces dependent tree nodes and evicts leaves first, even if they have more recent timestamps, preventing cascading recomputation (NVIDIA Developer) — yielding ~**20% higher cache hit rates** (NVIDIA Developer) versus standard LRU. For distributed serving, TensorRT-LLM supports full prefill-decode disaggregation with KV cache transfer via UCX, NIXL, or MPI, and integrates with NVIDIA Dynamo for KV-cache-aware routing at the data-center scale. (NVIDIA Developer)

Mooncake: KVCache as a first-class distributed resource

Mooncake, from Moonshot AI (FAST 2025 Best Paper), takes a fundamentally different approach: **KVCache is the central organizing principle** of the entire serving architecture. (arXiv) Rather than adding distributed caching to an existing engine, Mooncake builds a **disaggregated KVCache pool** that leverages underutilized CPU DRAM and SSD across the GPU cluster. (arXiv) A global **Conductor** scheduler dispatches requests based on current KVCache distribution, selecting prefill-decode node pairs and managing hot block replication across nodes. (arXiv)

The architecture works in four steps per request: (1) reuse existing cached prefix blocks, (2) perform prefill (arXiv) with **layer-wise streaming** that overlaps computation and KV cache transfer, (3) stream computed KV cache from prefill to decode node via the RDMA-based **Transfer Engine** (arXiv) (GPUDirect RDMA with topology-aware path selection), and (4) decode. Each KV cache block carries a hash value determined by both its content and prefix, enabling distributed deduplication. (arXiv) (arXiv) In production at Moonshot AI, Mooncake operates across **thousands of nodes** processing over **100 billion tokens daily** (ACM Digital Library) for Kimi, handling **75–525% more requests** than baseline configurations. (alphaXiv)

Critically, Mooncake's Transfer Engine has become a de facto standard for KV cache movement — integrated into SGLang (HiCache L3 backend), vLLM (KV Connector), TensorRT-LLM, and LMDeploy as of early 2026. It joined the PyTorch Ecosystem in February 2026. (Kvcache-ai) (GitHub)

MemServe: global prompt trees with unified memory pool

MemServe introduced **MemPool**, an elastic memory pool managing all cluster memory (CPU DRAM + GPU HBM) across instances through unified APIs. (arXiv) (ResearchGate) Its key innovation is **global prompt trees** — distributed radix trees with an extra field per node pointing to the instance storing the KV cache. The global scheduler tokenizes incoming requests, queries all tree types (prefill-only, decode-only, colocated) concurrently, and routes to the best instance. (arXiv) MemServe was the **first system to combine context caching with disaggregated inference**, (arXiv) showing that caching with disaggregation improves job completion time by up to **42%**. (arXiv) It remains a research prototype but influenced designs in LMCache and Mooncake.

Other frameworks

LMDeploy supports automatic prefix caching and gained PD disaggregation via Mooncake (June 2025), with up to 1.8× higher throughput than vLLM on certain workloads. (GitHub) **DeepSpeed-FastGen** does not implement automatic prefix caching; its focus is on efficient batching via Dynamic SplitFuse. **Ollama** uses slot-based KV caching without paged attention or cross-request sharing — designed for consumer-grade single-machine inference. (VMware)

Research frontier: from prefix matching to position-independent caching

The most significant limitation of RadixAttention — and all systems that followed it — is the requirement for **exact prefix matching**. Text must appear from position zero in the same order for KV cache reuse. Several research papers from 2024–2025 address this and other challenges.

Preble (ICLR 2025, UC San Diego) was the first system to extend prefix-aware scheduling to multi-GPU clusters. (arXiv) (ICLR) It maintains a **global radix tree** tracking which GPUs cache which prefix nodes, and uses an **E2 (Exploitation + Exploration) algorithm** that dynamically chooses between routing to the GPU with the longest cached prefix match versus the GPU with lowest load. (The Moonlight) A two-level hierarchical scheduler handles global request-level and per-GPU iteration-level scheduling. (arXiv) Performance: **1.5–14.5× average latency improvement** over single-node SGLang on shared-prompt workloads. (Wuklab)

CacheBlend (EuroSys 2025 Best Paper, University of Chicago) breaks the prefix-only barrier. (arXiv) In RAG scenarios where multiple retrieved chunks are concatenated, only the first chunk matches as a prefix — all others must be recomputed. (arXiv) CacheBlend reuses pre-computed KV caches for **all chunks** regardless of position, then selectively recomputes KV values for just **5–18% of tokens** to restore cross-attention relationships. It identifies which tokens need recomputation based on attention pattern divergence at each layer. (arXiv) Result: **2.2–3.3× TTFT reduction** and **2.8–5× throughput** increase (arXiv) with near-100% cache hit rates in RAG applications. (LMCache Blog)

EPIC (ICML 2025) extends position-independent caching further using a compilation/linking analogy — pre-generate chunk KV ("compile"), then selectively recompute tokens when concatenating ("link") using a LegoLink algorithm that exploits static attention sparsity. It achieves up to **8× TTFT improvement** and **7×**

throughput with 0–7% accuracy loss. (arXiv) (The Moonlight) **MEPIC** (December 2025) builds on EPIC to jointly manage prefix KV and chunk KV within shared HBM, reducing recomputation to block-level granularity. (arXiv)

CacheGen (SIGCOMM 2024, University of Chicago) addresses the network bottleneck for distributed KV cache by compressing KV tensors using layer-wise adaptive quantization and arithmetic coding, achieving **3.5–4.3× compression** with negligible quality loss. (arXiv +2) This is complementary to prefix caching — it makes cross-node KV transfer practical under bandwidth constraints.

Learned Prefix Caching (LPC) (NeurIPS 2025) replaces LRU eviction with an ML model that predicts conversation continuation probability, achieving **18–47% reduction in required cache size** for equivalent hit ratios. (NeurIPS)

Comparative analysis across key dimensions

Dimension	SGLang (HiCache)	vLLM + llm-d	TensorRT-LLM + Dynamo	Mooncake	MemServe
Cache data structure	Radix tree (HiRadixTree)	Hash table (block-level)	Radix search tree	Hash-based paged blocks	Global prompt trees (radix)
Tree-structured sharing	Native (multi-level branching)	No (linear prefix only)	Yes (tree dependencies tracked)	Via hash-based dedup	Yes (distributed radix trees)
Multi-node caching	HiCache L3 via Mooncake/NIXL/3FS	llm-d global index + LMCache	Dynamo KV events + NIXL	Native (core architecture)	MemPool (research prototype)
Eviction policy	LRU / LFU / write-through-selective	LRU	Priority-based + tree-aware	LRU / LFU / request-based	LRU
Cache-aware routing	Model Gateway (Rust, 474K ops/s)	llm-d scheduler (87.4% hit rate)	Dynamo Smart Router	Conductor (global scheduler)	Global scheduler with prompt trees
KV transfer mechanism	Mooncake RDMA / NIXL / Uni-PD (NVSHMEM)	KV Connector (Mooncake, NIXL, LMCache)	UCX / NIXL / MPI	GPUDirect RDMA (Transfer Engine)	MemPool transfer APIs

Dimension	SGLang (HiCache)	vLLM + llm-d	TensorRT-LLM + Dynamo	Mooncake	MemServe
PD disaggregation	Yes (EPD with heterogeneous TP)	Yes (via KV Connector)	Yes (native, 3 transfer backends)	Yes (core design, layer-wise streaming)	Yes (first caching + disagg system)
Position-independent reuse	No (prefix only; CacheBlend via LMCache)	No (prefix only; CacheBlend via LMCache)	No	No	No
Production scale	400K+ GPUs, trillions tokens/day	Massive (via llm-d, NIM)	Enterprise (Bing, NAVER)	1000s nodes, 100B+ tokens/day	Research prototype

What best-in-class looks like in early 2026

The optimal distributed prefix caching deployment in early 2026 is a **layered stack** rather than a single system. At the engine layer, **SGLang with HiCache** leads for multi-turn, agentic, and tree-structured workloads due to native radix-tree sharing and hierarchical caching, while **vLLM** remains competitive for high-concurrency batch inference with templated prompts. Runpod **LMCache** has emerged as the dominant middleware layer — used across vLLM Production Stack, NVIDIA Dynamo, llm-d, and KServe — providing standardized cross-engine KV cache management Lmcache GitHub with configurable chunk sizes and zero-copy tier traversal. arXiv For distributed storage, **Mooncake's Transfer Engine** is the de facto RDMA transport, integrated into all major engines, Kvcache-ai while **NVIDIA NIXL** provides the enterprise alternative with unified API across heterogeneous storage NVIDIA Developer (GPU Direct Storage, 3FS, S3).

Three critical gaps remain. First, **position-independent caching** (CacheBlend, EPIC) is still maturing OpenReview — no production system natively supports it, yet it offers the largest potential gains for RAG workloads where shared content doesn't align as exact prefixes. Second, **eviction intelligence** beyond LRU is underexploited; learned policies like LPC (NeurIPS 2025) show 18–47% cache size reduction but are not yet integrated into production engines. OpenReview Third, **semantic-level caching** that matches approximate rather than exact prefixes remains largely academic. The field is consolidating rapidly around the SGLang/vLLM + LMCache + Mooncake/NIXL stack, but the jump from prefix-only to position-independent KV reuse may prove as transformative as RadixAttention's original contribution.